

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Тема: «Ахо-Корасик»

Студент гр. 4343

Толмачев И.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Поиск набора образцов

Задание.

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст (T , $1 \leq |T| \leq 100000$).

Вторая - число n ($1 \leq n \leq 3000$), каждая следующая из n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\}$ $1 \leq |p_i| \leq 75$

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Выход:

Все вхождения образцов из P в T .

Каждое вхождение образца в текст представить в виде двух чисел - i p

Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

Описание алгоритма и оценка его сложности.

Для эффективного решения задачи используется алгоритм Ахо-Корасик, позволяющий выполнить поиск всех вхождений набора образцов за время, линейное относительно суммарной длины образцов и текста плюс количество найденных вхождений. Алгоритм строится на основе конечного автомата.

Построение бора. По всем образцам строится бор — корневое дерево, в котором каждая вершина соответствует некоторому префиксу одного или нескольких образцов. Каждая вершина хранит: массив переходов *next* размера $|\Sigma|=5$; переход по символу s ведёт в дочернюю вершину, либо помечен как отсутствующий (-1); длину пути от корня до данной вершины —

depth; список номеров образцов, которые заканчиваются в этой вершине — терминальная вершина; длину образцов, одинаковую для всех образцов, оканчивающихся в этой вершине.

Первоначально бор содержит только корень, вершина 0. Для каждого образца p_k происходит последовательный спуск от корня: если переход по очередному символу отсутствует, создаётся новая вершина, иначе используется существующая. В конечной вершине сохраняется номер образца k и его длина.

Время построения бора — $O(\sum |p_i|)$, память — $O(\sum |p_i| * |\Sigma|)$ в случае представления переходов массивом, либо $O(\sum |p_i|)$ при использовании разреженных структур; в нашей реализации выбран массив, так как алфавит мал.

Построение автомата: суффиксные и конечные ссылки. На основе бора строится детерминированный автомат. Для каждой вершины вычисляются: суффиксная ссылка *fail* — указывает на вершину, соответствующую наибольшему собственному суффиксу строки, представленной данной вершиной, который также является префиксом какого-либо образца; конечная ссылка *out* — сжатая хорошая суффиксная ссылка, указывающая на ближайшую терминальную вершину, достижимую по цепочке суффиксных ссылок. Если сама вершина терминальная, конечная ссылка обычно не используется для неё самой, но в реализации удобно класть её в суффиксную ссылку, если та терминальная, либо перенимать у суффиксной ссылки.

Вычисление ссылок выполняется обходом бора в ширину, начиная с корня. Для корня суффиксная ссылка направлена на самого себя. Для каждого символа c , если из корня есть переход, для дочерней вершины *fail* = 0 (корень), иначе переход перенаправляется в корень, так строится полный автомат. Затем для каждой вершины, извлекаемой из очереди: конечная ссылка: если *fail* является терминальной, то *out* = *fail*, иначе *out* = *fail.out*; для

каждого символа c : если переход из вершины существует, то для дочерней вершины $fail$ устанавливается в $fail.next[c]$, и она добавляется в очередь; если перехода нет, он перенаправляется в $fail.next[c]$.

После завершения BFS автомат становится полным: из любого состояния для любого символа существует переход, явный или перенаправленный. Сложность этапа $O(|\Sigma| * V)$, где $V \leq \sum |p_i| + 1$ — число вершин бора. Память остаётся $O(V * |\Sigma|)$ для хранения переходов и $O(V)$ для ссылок.

Поиск в тексте. Поиск осуществляется однократным чтением символов текста. Автомат начинает работу в состоянии 0 (корень). Для каждого символа текста t_i , 0-индекс i , выполняется переход $current = next[current][char_to_idx(t_i)]$. Затем для текущего состояния $current$ и всех вершин, достижимых по конечным ссылкам, собираются вхождения. Благодаря тому, что конечная ссылка указывает сразу на ближайшую терминальную вершину, обход цепочки суффиксных ссылок отсутствует; каждая обработанная терминальная вершина даёт одно или несколько вхождений. Для каждого найденного образца с длиной L и позицией конца i , 0-индекс, начало в тексте вычисляется как $start = i - L + 2$ (перевод в 1-индексацию). Полученные пары $(start, p_idx)$ сохраняются.

После обработки всего текста результаты упорядочиваются по позиции начала, а внутри одной позиции — по номеру образца.

Время поиска составляет $O(|T| + k)$, где k — общее количество вхождений. Каждый переход по конечной ссылке ведёт непосредственно к терминальной вершине, поэтому накладные расходы на холостые переходы исключены.

Оценка сложности по времени. Суммарная временная сложность алгоритма — $O(|T| + \sum |p_i| + k)$, что при заданных ограничениях, суммарная длина образцов до 225 000, длина текста до 100 000, позволяет обработать

данные за доли секунды.

Оценка сложности по памяти. Бор и автомат: для каждой вершины хранится массив переходов из 5 целых чисел, суффиксная и конечная ссылки, список номеров образцов, длина образца. При максимальном числе вершин до 225 001 и 4-байтовых целых это даёт примерно $225000 \cdot (5 \cdot 4 + 4 + 4) \approx 6.5$ МБ. Дополнительные структуры: массив вхождений может быть до $O(|T| + k)$ элементов. В худшем случае, когда текст состоит из одного повторяющегося символа, а образцы имеют длину 1, вхождений порядка $|T| \cdot n \leq 100000 \cdot 3000 = 3 \cdot 10^8$, что слишком велико. Однако по условию задачи суммарная длина образцов ограничена, а реальное число вхождений не может превысить суммарную длину текста и образцов в силу структуры бора; на практике выходной размер лимитирован. Итоговая потребность в памяти составляет $O(\sum |p_i| \cdot |\Sigma| + |T| + k)$, что в заданных границах приемлемо.

Тестирование.

Для проверки корректности алгоритма точного множественного поиска было проведено автоматическое тестирование с использованием фреймворка Google Test. Тесты охватывают как типичные ситуации, так и граничные случаи, включая работу с полным пятибуквенным алфавитом {A, C, G, T, N}. Каждый тест подаёт на вход текст и набор образцов, после чего сравнивает полученный список пар — позиция, номер образца — с ожидаемым.

Basic — базовая проверка: два образца "AC" и "TG" ищутся в тексте "ACTG". Ожидаются вхождения на позициях 1 и 3 соответственно.

SinglePatternFullText — единственный образец полностью совпадает с текстом, проверяется корректность при полном наложении.

NoMatch — текст "GG" не содержит ни одного из образцов "A" или "C", результат должен быть пустым.

OverlappingPatterns — в тексте "AAA" ищутся образцы "AA" длина 2 и

"А" длина 1. Проверяется корректное обнаружение множественных пересекающихся и вложенных вхождений.

PatternN — используется символ 'N', входящий в алфавит; образец и текст состоят только из него, что подтверждает работоспособность для всех допустимых символов.

MultiplePatternsSame — два одинаковых образца "А" с разными номерами. Тест гарантирует, что оба образца независимо фиксируются на одной и той же позиции.

Все перечисленные тесты выполняются успешно, что подтверждает правильность реализации алгоритма Ахо-Корасик для поиска набора подстрок в строке.

Точный поиск одного образца с джокером

Задание.

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с джокером.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблон образцу P необходимо найти все вхождения P в текст T .

Например, образец $ab??c?$ с джокером $?$ встречается дважды в тексте $xabvccbababcsax$.

Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы.

Все строки содержат символы из алфавита $\{A,C,G,T,N\}$

Вход:

Текст ($T, 1 \leq |T| \leq 100000$)

Шаблон ($P, 1 \leq |P| \leq 40$)

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

Описание алгоритма и оценка его сложности.

В основе лежит сведение задачи к множественному поиску безмасочных фрагментов образца. Алгоритм состоит из следующих этапов.

Разбиение образца на фрагменты. Образец P сканируется слева направо. Встречая джокер, завершаем текущий накапливаемый фрагмент. Таким

образом, образец разбивается на k непустых фрагментов $Q_1, Q_2, \dots, Q_k$, не содержащих джокеров. Для каждого фрагмента Q_i определяется его смещение l_i — позиция первого символа фрагмента относительно начала образца. Например, для образца $ab?c?$ фрагментами будут ab смещение 0 и c смещение 3.

Если образец начинается или заканчивается джокером, крайние фрагменты могут иметь ненулевые смещения; случай, когда весь образец состоит из джокеров, исключён условием задачи.

Данный шаг выполняется за $O(|P|)$.

Построение автомата Ахо-Корасик по фрагментам. Все фрагменты Q_i рассматриваются как набор образцов. Строится бор: каждая вершина дополнительно хранит глубину $depth$, равную длине фрагмента. В терминальных вершинах сохраняется список смещений l_i для тех фрагментов, которые заканчиваются в этой вершине, поскольку несколько одинаковых фрагментов могут иметь разные смещения.

Затем, как и в классическом алгоритме, вычисляются суффиксные *fail* и конечные *out* ссылки. Конечная ссылка указывает на ближайшую терминальную вершину по цепочке суффиксных ссылок, что гарантирует эффективный сбор вхождений. Время построения автомата — $O(\sum |Q_i| \cdot |\Sigma|) = O(|P|)$.

Поиск и подсчёт подтверждений. Создаётся массив счётчиков C размера $|T|+2$, изначально заполненный нулями. Индекс в массиве соответствует потенциальной начальной позиции образца в тексте.

Автомат обрабатывает символы текста последовательно. Когда на позиции i обнаруживается конец некоторого фрагмента Q_j , т.е. текущее состояние или вершина, достижимая по конечным ссылкам, является терминальной с данным фрагментом, вычисляется предполагаемое начало всего образца: $start = i - |Q_j| - l_j + 2$. Если $1 \leq start \leq |T| - |P| + 1$, то значение $C[start]$

увеличивается на единицу. Эта операция выполняется для каждого найденного вхождения каждого фрагмента.

После обработки всего текста позиции pos , для которых $C[pos] = k$, являются искомыми вхождениями образца, поскольку каждый из k фрагментов подтвердил своё присутствие на правильном относительном расстоянии.

Время поиска — $O(|T|+a)$, где a — количество вхождений всех фрагментов.

Вывод результата. Позиции, набравшие k подтверждений, выводятся в порядке возрастания. При ограничениях задачи таких позиций не более $|T|$.

Оценка сложности по времени. Общая временная сложность алгоритма — $O(|T|+|P|+a)$, что является квазилинейным.

Оценка сложности по памяти. Бор и автомат: $O(\sum |Q_i| \cdot |\Sigma|) = O(|P|)$ памяти для хранения вершин с переходами, фиксированный массив из 5 элементов. Массив счётчиков C : $O(|T|)$ памяти. Дополнительные структуры: списки смещений в вершинах, очередь для BFS — $O(|P|)$. Итоговая потребность в памяти — $O(|T|+|P|)$, что для максимальных длин $100\,000 + 40$ составляет около 400 КБ и полностью удовлетворяет ограничениям.

Тестирование.

Для проверки корректности алгоритма поиска образца с джокером также использовался фреймворк Google Test. Тесты покрывают различные варианты расположения джокеров, работу с алфавитом $\{A, C, G, T, N\}$ и случаи отсутствия вхождений. Каждый тест задаёт текст, образец и символ-джокер, после чего сверяет полученный список позиций с ожидаемым.

Basic — пример из условия: текст "ACTANCA", образец "A\$\$A\$" с джокером '\$'. Ожидается единственное вхождение на позиции 1, подтверждающее правильность разбиения на фрагменты и подсчёта

подтверждений.

WildcardAtStart — джокер в начале образца: "?AC" и текст "AAC". Вхождение на позиции 1, проверяется корректность смещений для первого фрагмента.

WildcardAtEnd — джокер в конце образца: "AC?" и текст "ACT". Вхождение на позиции 1, проверяется обработка завершающего джокера.

MultipleWildcards — несколько джокеров подряд три символа '?' в образце "A???A" и текст "ACTGA". Тест подтверждает, что цепочки масок любой длины корректно обрабатываются, а позиция определяется верно.

NoMatch — текст "GGG" не содержит ни одного из фрагментов образца "A?C", результат пуст. Проверяется реакция на полное отсутствие вхождений.

SingleWildcard — образец "A?" с одним джокером и текст "AN". Символ 'N' покрывается джокером, вхождение находится на позиции 1.

OnlyFragmentsNoWildcards — образец "CT" без джокеров, символ джокера '*' не встречается в образце. Тест показывает, что алгоритм вырождается в обычный точный поиск и выдаёт позицию 2 в тексте "ACTG".

Все тесты отрабатывают успешно, что подтверждает корректность реализации алгоритма поиска шаблонов с масками на основе Ахо-Корасик.